

Work Integrated Learning Programmes Division

M. Tech. in Artificial Intelligence & Machine Learning

1st Semester 2026-27

Assignment I

AIMLZC416 — Mathematical Foundations for Machine Learning

2026aj05009 - Saurabh Tayal

Q1) Finding solutions of linear systems

1. Write a code taking as input a matrix A of size $m \times n$ and a vector b of size $m \times 1$, where m and n are arbitrarily large numbers and $m < n$, constructing the augmented matrix and performing $\square$ REF, and $\square$ RREF without using any built-in functions. In case you encounter any division by 0, you can choose a different A and/or b. Note that this part should be explicitly there in the code and give a comment line on the same. (1 mark + 1 mark)

```

1 import numpy as np
2
3 # Q1(1): Define reusable functions to calculate REF and RREF of [A | b].
4 # A has m rows and n coefficient columns, and b has m rows and one column.
5 # Appending b to A creates the augmented matrix [A | b] with shape m x (n + 1).
6 # The row operations below are written explicitly to show the elimination logic.
7 # This is a manual Gaussian-elimination approach, which is expected in the
  assignment.
8 # It helps us understand how pivoting and elimination preserve the solution set
  while converting the matrix into a simpler triangular form.
9
10
11 def matrix_multiply(first_matrix, second_matrix):
12     # If the first matrix is p x q and the second is q x r, their product is p
      x r. Each output entry is the sum of q corresponding products.
13     # This function is used repeatedly throughout the notebook to perform
      matrix multiplication without built-in functions.
14     first_rows = len(first_matrix)
15     first_columns = len(first_matrix[0])
16     second_rows = len(second_matrix)
17     second_columns = len(second_matrix[0])
18
19     if first_columns != second_rows:
20         raise ValueError("The matrices cannot be multiplied: inner dimensions
          differ.")
21
22     product = np.zeros((first_rows, second_columns))
23     for i in range(first_rows):
24         for j in range(second_columns):
25             total = 0.0
26             for k in range(first_columns):
27                 total += first_matrix[i, k] * second_matrix[k, j]
28             product[i, j] = total
29     return product
30
31
32 def transpose_matrix(matrix):
33     # The transpose changes rows into columns. The original entry at (i, j) is
      copied to position (j, i) in the transposed matrix.
34     # This is useful in later tasks where we compute  $C = X^T X$  and project
      vectors onto eigen-directions.
35     rows = len(matrix)
36     columns = len(matrix[0])
37     transposed = np.zeros((columns, rows))
38     for i in range(rows):
39         for j in range(columns):
40             transposed[j, i] = matrix[i, j]
41     return transposed
42
43
44 def ref_form(augmented_matrix, tolerance=1e-12):
45     # Use a floating-point copy so row operations do not modify the input.
46     # REF (Row Echelon Form) keeps all entries below each pivot equal to zero.
47     # This is the first stage of Gaussian elimination and is needed to identify
      pivot positions.

```

```

48     ref_matrix = augmented_matrix.copy().astype(float)
49     number_of_rows, number_of_columns = ref_matrix.shape
50     coefficient_columns = number_of_columns - 1
51     pivot_row = 0
52
53     # Scan coefficient columns from left to right. 'pivot_row' means the next
54     # row where a successful pivot must be placed; it changes only after a pivot.
55     for column in range(coefficient_columns):
56         # If all rows already contain pivots, no further elimination is needed.
57         if pivot_row == number_of_rows:
58             break
59
60         # Search the current row and all rows below it for a non-zero entry.
61         # Treat very small floating-point values as zero using the tolerance.
62         candidate_row = pivot_row
63         while (
64             candidate_row < number_of_rows
65             and abs(ref_matrix[candidate_row, column]) < tolerance
66         ):
67             candidate_row += 1
68
69         # If no usable entry is found, the column has no pivot. Skip it instead
70         # of dividing by zero; keep pivot_row unchanged for the next column.
71         if candidate_row == number_of_rows:
72             continue
73
74         # Move the usable row into the next pivot position. Row swaps preserve
75         # the solution set while placing a non-zero pivot where it is needed.
76         ref_matrix[[pivot_row, candidate_row]] = ref_matrix[[candidate_row,
77             pivot_row]]
78
79         # The pivot is non-zero because of the search above, so division is
80         # safe.
81         # Scale the complete row so that its pivot entry becomes exactly 1.
82         ref_matrix[pivot_row] = ref_matrix[pivot_row] / ref_matrix[pivot_row,
83             column]
84
85         # Subtract a multiple of the pivot row from each lower row. This makes
86         # every entry below the pivot zero and creates the REF structure.
87         for next_row in range(pivot_row + 1, number_of_rows):
88             multiplier = ref_matrix[next_row, column]
89             ref_matrix[next_row] = (
90                 ref_matrix[next_row] - multiplier * ref_matrix[pivot_row]
91             )
92
93         # A pivot was successfully created, so the next pivot belongs in the
94         # next row. If a column had no pivot, this increment would not occur.
95         pivot_row += 1
96
97     return ref_matrix
98
99 def rref_form(augmented_matrix, tolerance=1e-12):
100     # REF has zeros below each pivot. Begin with REF and remove entries above
101     # each pivot to obtain Reduced Row Echelon Form.
102     # RREF makes the solution structure obvious because each pivot column
103     # contains only one non-zero entry, equal to 1.
104     rref_matrix = ref_form(augmented_matrix, tolerance)
105     number_of_rows, number_of_columns = rref_matrix.shape
106     coefficient_columns = number_of_columns - 1
107
108     # Process rows from bottom to top. This ensures that eliminating an entry
109     # above a pivot does not disturb pivots that have already been handled below.
110     for pivot_row in range(number_of_rows - 1, -1, -1):
111         # The first non-zero coefficient in this row identifies its pivot
112         # column.
113         pivot_column = 0
114         while (
115             pivot_column < coefficient_columns
116             and abs(rref_matrix[pivot_row, pivot_column]) < tolerance
117         ):
118             pivot_column += 1
119
120         # If no coefficient is non-zero, this is an all-zero row with no pivot.
121         if pivot_column == coefficient_columns:
122             continue
123
124         # The pivot was normalized to 1 during REF. Eliminate its entries from
125         # every row above so the pivot is the only non-zero value in its column.
126         for previous_row in range(pivot_row):
127             multiplier = rref_matrix[previous_row, pivot_column]
128             rref_matrix[previous_row] = (
129                 rref_matrix[previous_row] - multiplier * rref_matrix[pivot_row]

```

```

118         )
119
120     return rref_matrix
121
122
123     print("Assuming the size of A = m x n, where m < n, as 4 x 5 for this example.")
124     m = 4
125     n = 5
126
127     # Generate reproducible random decimal entries as required by the assignment.
128     # We intentionally choose m < n so the system is underdetermined and therefore
129     # has free variables.
130     SEED = 2026
131     np.random.seed(SEED)
132     A = np.random.uniform(0.1, 9.9, size=(m, n))
133     # Generate an independent random right-hand-side vector b.
134     # This is the target vector we want to solve for: A x = b.
135     b = np.random.uniform(0.1, 9.9, size=(m, 1))
136     augmented = np.hstack((A, b))
137
138     print("\nMatrix A =")
139     print(A)
140     print("\nVector b =")
141     print(b)
142     print("\nAugmented matrix [A | b] =")
143     print(augmented)
144
145     # Compute and display REF and RREF. The functions are reused in Q1.2 and Q1.3.
146     # These outputs are the symbolic evidence that the row operations preserve the
147     # solution set.
148     ref = ref_form(augmented)
149     rref = rref_form(augmented)
150
151     print("\nREF =")
152     print(ref)
153     print("\nRREF =")
154     print(rref)

```

Assuming the size of A = m x n, where m < n, as 4 x 5 for this example.

```

Matrix A =
[[2.24958722 4.14751502 9.67102769 0.97121041 4.79704062]
 [9.77799484 2.04178267 9.03480878 5.57085113 7.77564879]
 [3.06427058 3.99354974 3.00047968 8.86206692 0.57211601]
 [4.4616629 5.60656011 3.66440338 6.90603218 9.1490072 ]]

Vector b =
[[0.39981206]
 [1.2897733 ]
 [2.37268242]
 [6.40039011]]

Augmented matrix [A | b] =
[[2.24958722 4.14751502 9.67102769 0.97121041 4.79704062 0.39981206]
 [9.77799484 2.04178267 9.03480878 5.57085113 7.77564879 1.2897733 ]
 [3.06427058 3.99354974 3.00047968 8.86206692 0.57211601 2.37268242]
 [4.4616629 5.60656011 3.66440338 6.90603218 9.1490072 6.40039011]]

REF =
[[ 1.          1.84367824  4.29902321  0.43172828  2.13240926  0.17772685]
 [-0.          1.          2.06440951 -0.08441387  0.81792121  0.0280275 ]
 [-0.         -0.          1.         -1.09550634  0.68219089 -0.27752693]
 [-0.         -0.         -0.          1.         -1.37331156 -0.4552538 ]]

RREF =
[[ 1.          0.          0.          0.          1.83636994  0.77608257]
 [ 0.          1.          0.          0.          2.39951861  1.59211701]
 [-0.         -0.          1.          0.         -0.82228063 -0.77626035]
 [-0.         -0.         -0.          1.         -1.37331156 -0.4552538 ]]

```

2. Write a Python code to identify the pivot and non-pivot columns and find the particular solution and solutions to $Ax = 0$. (1 mark)

```

1 # Q1(2): Continue with A, b, rref_form, and matrix_multiply from part 1.1.
2 # This cell also defines the consistency check needed before solving Ax = b.
3 # The RREF identifies pivot variables and free variables.
4 # The logic is: first reduce the system, then classify variables into pivot/free, then construct one solution and th
5
6
7 def has_inconsistent_row(rref_matrix, coefficient_columns, tolerance=1e-12):
8     # In the RREF of [A | b], a row of the form [0 0 ... 0 | c], where c is non-zero, represents 0 = c.
9     # This contradiction means that the system Ax = b has no solution.
10    rows = len(rref_matrix)

```

```

11     for row in range(rows):
12         coefficients_are_zero = True
13
14         # Check whether every coefficient in this row is effectively zero.
15         # The tolerance handles tiny round-off values produced by floating-point arithmetic.
16         for column in range(coefficient_columns):
17             if abs(rref_matrix[row, column]) >= tolerance:
18                 coefficients_are_zero = False
19                 break
20
21         # If all coefficients are zero but the augmented value is non-zero, the row is inconsistent and the function
22         if (
23             coefficients_are_zero
24             and abs(rref_matrix[row, coefficient_columns]) >= tolerance
25         ):
26             return True
27
28     # No contradictory row was found, so the system is consistent.
29     return False
30
31
32 # Recalculate RREF from the same augmented matrix created in part 1.1.
33 rref = rref_form(augmented)
34
35 # A consistency test must happen before extracting solutions. If the system is inconsistent, no particular solution
36 if has_inconsistent_row(rref, n):
37     print("The system is inconsistent and has no solution.")
38     raise SystemExit("Program stopped because no solution exists.")
39
40 # Identify pivot columns by scanning each RREF row from left to right.
41 # The first non-zero coefficient in a non-zero row is that row's pivot.
42 # A pivot column corresponds to a dependent variable that is solved directly by the system.
43 pivot_columns = []
44 for row in range(m):
45     for column in range(n):
46         if abs(rref[row, column]) > 1e-12:
47             pivot_columns.append(column)
48             break
49
50 # Every coefficient column that is not a pivot column represents a free variable.
51 # These variables can be assigned arbitrary values.
52 non_pivot_columns = []
53 for column in range(n):
54     if column not in pivot_columns:
55         non_pivot_columns.append(column)
56
57 print("Pivot columns (0-based):", pivot_columns)
58 print("Non-pivot columns (0-based):", non_pivot_columns)
59
60 # Choose zero for every free variable. Reading the augmented column then gives one valid assignment for the pivot va
61 # This is the special solution x_p that satisfies A x_p = b.
62 particular_solution = np.zeros(n)
63 for row, pivot_column in enumerate(pivot_columns):
64     # 'row' identifies the RREF equation, while 'pivot_column' identifies the actual variable solved by that equatio
65     particular_solution[pivot_column] = rref[row, n]
66
67 print("\nParticular solution x_p =")
68 print(particular_solution)
69 print("Check A x_p =")
70 # Reshape x_p into an n x 1 column vector because A has shape m x n.
71 # The manual multiplication returns an m x 1 vector, which is flattened for display.
72 print(matrix_multiply(A, particular_solution.reshape(n, 1)).reshape(m))
73 print("Check b =")
74 print(b.reshape(m))
75
76 # Construct homogeneous solutions for Ax = 0. Process one free variable at a time:
77 # set the selected free variable to 1 and all other free variables to 0.
78 # Each such vector is a basis vector for the null space of A.
79 null_space_vectors = []
80 for free_column in non_pivot_columns:
81     vector = np.zeros(n)
82     vector[free_column] = 1
83
84     # For each RREF equation, solve the pivot variable in terms of the selected free variable. The minus sign comes
85     for row, pivot_column in enumerate(pivot_columns):
86         vector[pivot_column] = -rref[row, free_column]
87     null_space_vectors.append(vector)
88
89 print("\nVectors for solutions of A x = 0:")
90 for number, vector in enumerate(null_space_vectors, start=1):
91     print("v" + str(number) + " =", vector)
92     print("A v" + str(number) + " =")

```

```

93 # Each result should be the zero vector, apart from insignificant floating-point round-off errors such as 1e-15.
94 print(matrix_multiply(A, vector.reshape(n, 1)).reshape(m))
95
96 # The complete solution of  $Ax = b$  is one particular solution plus any linear combination of the homogeneous (null-sp
97 # This is the standard structure of solutions to a consistent linear system with free variables.
98 print("\nGeneral solution:")
99 print("x = x_p + c1*v1 + c2*v2 + ...")
100 print("Actual general solution:")
101 print(
102     "x = " + np.array2string(particular_solution, precision=6, separator=", ", end=""
103 )
104 for i, vector in enumerate(null_space_vectors, start=1):
105     print(
106         " + c" + str(i) + "*" + np.array2string(vector, precision=6, separator=", ",
107         end="",
108     )
109 print()
110
111 # Verification step: choose random values for the free parameters and substitute them into the general solution.
112 # If the theory is correct,  $A * x_{\text{general}}$  should still equal  $b$  because each null-space vector satisfies  $A * v_i = 0$ .
113 print("\nVerification of the general solution:")
114 print("This is the verification step using random values of the free parameters.")
115 random_coefficients = np.random.uniform(-2, 2, size=len(null_space_vectors))
116 x_general = particular_solution.copy()
117 for coefficient, vector in zip(random_coefficients, null_space_vectors):
118     x_general = x_general + coefficient * vector
119
120 print("Random coefficients c:", random_coefficients)
121 print("x_general =")
122 print(x_general)
123 print("A x_general =")
124 print(matrix_multiply(A, x_general.reshape(n, 1)).reshape(m))
125 print("b =")
126 print(b.reshape(m))
127 print(
128     "This verifies the general solution because  $A x_{\text{general}} = b$  for the chosen random coefficients."
129 )

```

Pivot columns (0-based): [0, 1, 2, 3]
 Non-pivot columns (0-based): [4]

Particular solution x_p =
 [0.77608257 1.59211701 -0.77626035 -0.4552538 0.]
 Check $A x_p$ =
 [0.39981206 1.2897733 2.37268242 6.40039011]
 Check b =
 [0.39981206 1.2897733 2.37268242 6.40039011]

Vectors for solutions of $Ax = 0$:
 v_1 = [-1.83636994 -2.39951861 0.82228063 1.37331156 1.]
 $A v_1$ =
 [-8.88178420e-16 6.21724894e-15 -1.99840144e-15 0.00000000e+00]

General solution:
 $x = x_p + c_1 v_1 + c_2 v_2 + \dots$
 Actual general solution:
 $x = [0.776083, 1.592117, -0.77626, -0.455254, 0.] + c_1 [-1.83637, -2.399519, 0.822281, 1.373312, 1.]$

Verification of the general solution:
 This is the verification step using random values of the free parameters.
 Random coefficients c : [-1.0562656]
 x_{general} =
 [2.71577695 4.12664597 -1.64480709 -1.90583555 -1.0562656]

```
A x_general =
[0.39981206 1.2897733 2.37268242 6.40039011]
b =
[0.39981206 1.2897733 2.37268242 6.40039011]
This verifies the general solution because A x_general = b for the chosen random coefficients.
```

3. Consider a random 5×7 matrix A and a suitable b and show the REF, RREF, pivot columns, non-pivot columns, the particular solution, the solutions to $Ax = 0$, the general solution and verify the general solution. (1/4 $\times$ 8 = 2 marks)

```
1 # Q1(3): Continue using ref_form, rref_form, matrix_multiply, and has_inconsistent_row from parts 1.1 and 1.2.
2 # This cell applies the complete solution process to the required 5 x 7 example.
3 # A 5 x 7 system has 5 equations and 7 variables, so a full-row-rank example normally has 5 pivot variables and 2 fr
4 # This is the general pattern for a rectangular system with more variables than equations.
5
6 np.random.seed(SEED)
7 A_5x7 = np.random.uniform(0.1, 9.9, size=(5, 7))
8 # Generate b independently, as required. The solution will be obtained later
9 # from the RREF; it is not supplied in advance.
10 # This ensures we are solving a genuine system rather than assuming a solution beforehand.
11 b_5x7 = np.random.uniform(0.1, 9.9, size=(5, 1))
12 augmented_5x7 = np.hstack((A_5x7, b_5x7))
13
14 # Reuse the functions from Q1.1. Row reduction transforms the augmented system but preserves its solution set.
15 ref_5x7 = ref_form(augmented_5x7)
16 rref_5x7 = rref_form(augmented_5x7)
17
18 print("A =")
19 print(A_5x7)
20 print("\nb =")
21 print(b_5x7.reshape(5))
22 print("\nREF of [A | b] =")
23 print(ref_5x7)
24 print("\nRREF of [A | b] =")
25 print(rref_5x7)
26
27 # Before solving, test for a row [0 0 ... 0 | c] with c non-zero.
28 # Such a row represents the contradiction 0 = c and proves that no solution exists.
29 if has_inconsistent_row(rref_5x7, 7):
30     print("\nThe 5 x 7 system is inconsistent and has no solution.")
31     raise SystemExit("Program stopped because no solution exists.")
32
33 # Scan only the seven coefficient columns. The final column is b and must not be treated as a variable or as a pivot
34 # This identifies which variables are pivot variables and which are free variables.
35 pivot_columns_5x7 = []
36 for row in range(5):
37     for column in range(7):
38         if abs(rref_5x7[row, column]) > 1e-12:
39             pivot_columns_5x7.append(column)
40             break
41
42 # Columns without pivots correspond to free variables. Their values determine the family of solutions because there
43 non_pivot_columns_5x7 = [
44     column for column in range(7) if column not in pivot_columns_5x7
45 ]
46
47 print("\nPivot columns:", pivot_columns_5x7)
48 print("Non-pivot columns:", non_pivot_columns_5x7)
49
50 # Set all free variables to zero. The augmented value in each pivot row then gives one particular solution of Ax = b
51 # This special solution is one member of the complete solution set.
52 particular_solution_5x7 = np.zeros(7)
53 for row, pivot_column in enumerate(pivot_columns_5x7):
54     particular_solution_5x7[pivot_column] = rref_5x7[row, 7]
55
56 print("\nParticular solution x_p =")
57 print(particular_solution_5x7)
58 print("Check A x_p =")
59 print(matrix_multiply(A_5x7, particular_solution_5x7.reshape(7, 1)).reshape(5))
60 print("Check b =")
61 print(b_5x7.reshape(5))
62
63 # Build one vector for the null space for each free variable.
64 # For each vector, one free variable is set to 1 and the other free variables are set to 0. The RREF equations deter
65 # These vectors form a basis for the null space of A: they satisfy A v = 0.
66 null_space_vectors_5x7 = []
67 for free_column in non_pivot_columns_5x7:
68     vector = np.zeros(7)
69     vector[free_column] = 1
70     for row, pivot_column in enumerate(pivot_columns_5x7):
71         vector[pivot_column] = -rref_5x7[row, free_column]
72     null_space_vectors_5x7.append(vector)
```

```

73
74 print("\nSolutions to A x = 0:")
75 for number, vector in enumerate(null_space_vectors_5x7, start=1):
76     print("v" + str(number) + " =", vector)
77     print("A v" + str(number) + " =")
78     # These products should be zero, with tiny numerical round-off values allowed.
79     print(matrix_multiply(A_5x7, vector.reshape(7, 1)).reshape(5))
80
81 # Since this example has two free variables, every solution is described by two arbitrary constants multiplying the
82 # The general solution is  $x = x_p + c_1 v_1 + c_2 v_2$ .
83 print("\nGeneral solution:")
84 print("x = x_p + c1*v1 + c2*v2")
85 print("Actual general solution:")
86 print(
87     "x = " + np.array2string(particular_solution_5x7, precision=6, separator=", "),
88     end="",
89 )
90 for i, vector in enumerate(null_space_vectors_5x7, start=1):
91     print(
92         " + c" + str(i) + "*" + np.array2string(vector, precision=6, separator=", "),
93         end="",
94     )
95 print()
96
97 # Verification step: choose random values for the free parameters and check that the full general solution still sat
98 # This confirms the theory: adding a null-space vector does not change the equation because  $A v = 0$ .
99 print("\nVerification of the general solution:")
100 print("This is the verification step using random values of the free parameters.")
101 random_coefficients_5x7 = np.random.uniform(-2, 2, size=len(null_space_vectors_5x7))
102 x_general_5x7 = particular_solution_5x7.copy()
103 for coefficient, vector in zip(random_coefficients_5x7, null_space_vectors_5x7):
104     x_general_5x7 = x_general_5x7 + coefficient * vector
105
106 print("Random coefficients c:", random_coefficients_5x7)
107 print("x_general =")
108 print(x_general_5x7)
109 print("A x_general =")
110 print(matrix_multiply(A_5x7, x_general_5x7.reshape(7, 1)).reshape(5))
111 print("b =")
112 print(b_5x7.reshape(5))
113 print(
114     "This verifies the general solution because  $A x_{\text{general}} = b$  for the chosen random coefficients."
115 )
116 print("The checks above verify  $A x_p = b$  and  $A v_i = 0$ .")
117

```

```

Particular solution x_p =
[-0.08380277  2.10996141 -0.36456778 -1.09384376  0.23018352  0.
  0.          ]
Check A x_p =
[5.07867715  5.72980397  4.90339048  2.130364    4.51865889]
Check b =
[5.07867715  5.72980397  4.90339048  2.130364    4.51865889]

Solutions to A x = 0:
v1 = [ 1.33034018 -4.72754021  0.5362321   2.55908938 -0.17396077  1.
  0.          ]
A v1 =
[-5.32907052e-15 -2.26485497e-14 -1.77635684e-15 -4.44089210e-16
 -7.99360578e-15]
v2 = [-0.99075339 -0.77080223  0.353124    0.73740619 -0.15579005  0.
  1.          ]
A v2 =
[-4.44089210e-16  1.77635684e-15 -3.33066907e-16  3.33066907e-16
 -1.77635684e-15]

General solution:
x = x_p + c1*v1 + c2*v2
Actual general solution:
x = [-0.083803,  2.109961, -0.364568, -1.093844,  0.230184,  0.
  0.          ] + c1*[ 1.33034 , -4.72754 ,  0.536232,  2.559089, -0.173961,  1.
  0.          ] + c2*[-0.990753, -0.770802,  0.353124,  0.737406, -0.15579 ,  0.
  1.          ]

Verification of the general solution:
This is the verification step using random values of the free parameters.
Random coefficients c: [ 1.75971544 -0.7529851 ]
x_general =
[ 3.00323993 -5.6287615  0.31315103  2.85416948  0.04136965  1.75971544
 -0.7529851 ]
A x_general =
[5.07867715  5.72980397  4.90339048  2.130364    4.51865889]
b =
[5.07867715  5.72980397  4.90339048  2.130364    4.51865889]
This verifies the general solution because A x_general = b for the chosen random coefficients.
The checks above verify A x_p = b and A v_i = 0

```

Q2) Consider a dataset $X \in \mathbb{R}^{500 \times 6}$ constructed as follows: the first four features f_1, f_2, f_3, f_4 are generated as random features sampled from a standard normal distribution (read about this). The fifth and sixth features are defined by the relations $f_5 = 2f_1 + 3f_2$, $f_6 = f_3 - 2f_4$. Perform the following tasks in sequence:

1. Write a Python code to generate the dataset $X = [f_1, f_2, f_3, f_4, f_5, f_6]$. [0.5]

```

1 # Q2(1): Generate the dataset X = [f1, f2, f3, f4, f5, f6].
2 # Each of the 500 rows represents one observation and each column represents one feature.
3 # The first four features are sampled independently from N(0, 1), the standard normal distribution.
4 # Features f5 and f6 are then deliberately created as linear combinations: f5 = 2f1 + 3f2 and f6 = f3 - 2f4.
5 # These exact relationships will later explain why the six-column matrix has rank 4 rather than rank 6.
6 # In other words, the last two columns do not add independent information; they are combinations of the first four columns.
7
8 import numpy as np
9
10 # Fixing the seed makes the experiment reproducible: every execution generates the same X.
11 np.random.seed(SEED)
12 number_of_data_points = 500
13
14 # randn generates independent samples from a standard normal distribution with mean 0 and variance 1.
15 # Keeping these four columns independent provides four base directions.
16 f1 = np.random.randn(number_of_data_points)
17 f2 = np.random.randn(number_of_data_points)
18 f3 = np.random.randn(number_of_data_points)
19 f4 = np.random.randn(number_of_data_points)
20
21 # These columns contain no new independent information because they are determined exactly by the first four columns.
22 # Their dependence is the key structure being studied in Q2.
23 f5 = 2 * f1 + 3 * f2
24 f6 = f3 - 2 * f4
25
26 # column_stack places the six equal-length feature arrays side by side. Thus X has shape (500, 6), and its fifth and sixth columns are f5 and f6.
27 X = np.column_stack((f1, f2, f3, f4, f5, f6))
28
29 print("Shape of X:", X.shape)
30 print("First five rows of X:")
31 print(X[:5])
32

```



```

Shape of X: (500, 6)
First five rows of X:
[[-0.43171852 -0.70255252 -0.10353308  0.87732991 -2.9710946 -1.85819289]
 [-1.39287397  1.50152579 -0.73937348  1.6569189  1.71882945 -4.05321129]
 [ 0.31157067 -0.60182461  1.21508673  1.27213227 -1.1823325 -1.32917782]
 [-0.01323488  1.44440069  1.47784444  1.3170826  4.3067323 -1.15632076]
 [ 1.44970773 -0.29466117 -0.19591321  0.71605898  2.01543195 -1.62803116]]

```

2. Write a Python code which computes the rank of X and display the output for the dataset X generated in step 1. [0.5]

```

1 # Q2(2): Compute rank(X) without calling a built-in rank function.
2 # The rank is the number of linearly independent columns, which is equal to the number of pivots produced by Gaussian elimination.
3 # Since f5 and f6 are exact combinations of f1-f4, the expected rank is 4. Row reduction verifies this numerically.
4 # The purpose of this task is to show the difference between the matrix dimension (6 columns) and the true number of independent columns.
5
6 # Work on a floating-point copy so row operations do not change the original dataset X.
7 reduced_X = X.copy().astype(float)
8 number_of_rows, number_of_columns = reduced_X.shape
9 rank_of_X = 0
10 tolerance = 1e-12
11
12 # Examine columns from left to right. rank_of_X is both the number of pivots found and the row where the next successful pivot is found.
13 for column in range(number_of_columns):
14     # Once every row has a pivot, no additional independent direction can be found.
15     if rank_of_X == number_of_rows:
16         break
17
18     # Search the current pivot row and all rows below it for a usable non-zero entry.
19     # A tolerance treats tiny floating-point round-off values as mathematical zeros.
20     pivot_row = rank_of_X
21     while pivot_row < number_of_rows and abs(reduced_X[pivot_row, column]) < tolerance:
22         pivot_row += 1
23
24     # A zero remainder in this column means that the column contributes no pivot.
25     # Continuing preserves the current pivot row for the next candidate column.
26     if pivot_row == number_of_rows:
27         continue
28
29     # Swap the usable row into position, then normalize it so the pivot equals 1.
30     # These elementary row operations preserve the rank and simplify elimination.
31     reduced_X[[rank_of_X, pivot_row]] = reduced_X[[pivot_row, rank_of_X]]
32     reduced_X[rank_of_X] = reduced_X[rank_of_X] / reduced_X[rank_of_X, column]
33
34     # Eliminate entries below the pivot. A successful pivot represents one independent row/column direction, so advance the rank.
35     for next_row in range(rank_of_X + 1, number_of_rows):
36         multiplier = reduced_X[next_row, column]
37         reduced_X[next_row] = reduced_X[next_row] - multiplier * reduced_X[rank_of_X]
38     rank_of_X += 1
39
40 print("Rank of X:", rank_of_X)
41

```

Rank of X: 4

3. Numerical Experiment with the Power Method Read about the power method for finding the dominant eigenvalue and its corresponding eigenvector and perform the following tasks.

(a) Write a Python code to compute the covariance matrix [0.5]

$C = (1/n)X^T X$ where n is number of data points

```

1 # Q2(3a): Compute the covariance/Gram matrix  $C = (1/n) X^T X$  explicitly.
2 # The functions transpose_matrix and matrix_multiply are already defined in previous cells and are being reused here.
3 # Keeping a single shared definition avoids duplicate function redefinitions while preserving the same manual linear-al
4 # This matrix captures pairwise similarities between features; because the data are centered in this formulation, it is
5
6 number_of_data_points = X.shape[0]
7 transposed_X = transpose_matrix(X)
8 # Multiplying  $X^T$  by  $X$  accumulates pairwise feature products; division by  $n$  converts each accumulated sum into the requ
9 # Since the matrix is symmetric,  $C = C^T$ , and its eigenvectors are orthogonal. This matters for the later power method
10 C = matrix_multiply(transposed_X, X) / number_of_data_points
11
12 print("n =", number_of_data_points)
13 print("Shape of  $X^T$  =", transposed_X.shape)
14 print("Covariance matrix C =")
15 print(C)
16

```

```

n = 500
Shape of  $X^T$  = (6, 500)
Covariance matrix C =
[[ 9.99410950e-01 -1.60051587e-02  1.89588472e-02  9.93510963e-03
   1.95080642e+00 -9.11372085e-04]
 [-1.60051587e-02  1.11925929e+00 -1.33242656e-01  6.06364469e-02
   3.32576755e+00 -2.54515550e-01]
 [ 1.89588472e-02 -1.33242656e-01  1.14325526e+00 -3.01520021e-02
  -3.61810275e-01  1.20355926e+00]
 [ 9.93510963e-03  6.06364469e-02 -3.01520021e-02  9.94448097e-01
   2.01779560e-01 -2.01904820e+00]
 [ 1.95080642e+00  3.32576755e+00 -3.61810275e-01  2.01779560e-01
   1.38789155e+01 -7.65369395e-01]
 [-9.11372085e-04 -2.54515550e-01  1.20355926e+00 -2.01904820e+00
  -7.65369395e-01  5.24165565e+00]]

```

(b) Implement the Power Method in Python to approximate the largest eigenvalue λ_1 and its corresponding eigenvector v_1 of C . Show the code and the outputs. [1]

```

1 # Q2(3b): Compute all eigenvalues and eigenvectors from the covariance matrix using the power method with deflation.
2 # The procedure is: find the dominant eigenpair of the current matrix, then deflate it and repeat.
3 # This gives all eigenpairs of a real symmetric matrix.
4 # The logic is based on the fact that for a symmetric matrix, applying the power method repeatedly causes the vector
5
6 np.set_printoptions(precision=8, suppress=False)
7
8
9 def vector_norm(vector):
10     # Euclidean norm of a vector. The relation  $\|x\| = \sqrt{\sum x_i^2}$  is required before normalization.
11     flattened_vector = vector.reshape(-1)
12     squared_length = 0.0
13     for value in flattened_vector:
14         squared_length += value * value
15     return np.sqrt(squared_length)
16
17
18 def power_method(matrix, tolerance=1e-7, maximum_iterations=10000):
19     # Power method iteration for the largest eigenvalue/eigenvector of a matrix.
20     # Step 1: start with a non-zero vector.
21     # Step 2: repeatedly multiply by A and normalize.
22     # Step 3: use the Rayleigh quotient to approximate the eigenvalue.
23     # Step 4: stop when the eigenvalue changes by less than tolerance.
24     vector = np.ones((matrix.shape[0], 1))
25     vector = vector / vector_norm(vector)
26     old_eigenvalue = 0.0
27
28     for iteration in range(1, maximum_iterations + 1):
29         new_vector = matrix_multiply(matrix, vector)
30         new_vector_length = vector_norm(new_vector)
31         if new_vector_length == 0:
32             raise ValueError("The power method reached a zero vector.")
33         vector = new_vector / new_vector_length
34
35         transpose_vector = transpose_matrix(vector)
36         matrix_vector = matrix_multiply(matrix, vector)
37         rayleigh_quotient = matrix_multiply(transpose_vector, matrix_vector)
38         eigenvalue = rayleigh_quotient[0, 0]
39

```

```

40     if abs(eigenvalue - old_eigenvalue) < tolerance:
41         return eigenvalue, vector.reshape(matrix.shape[0]), iteration
42     old_eigenvalue = eigenvalue
43
44     return eigenvalue, vector.reshape(matrix.shape[0]), maximum_iterations
45
46
47 def power_method_all(matrix):
48     # Repeatedly apply the power method to the current matrix after deflation.
49     # Deflation uses the identity  $A_k = A - \sum_{i=1}^k \lambda_i v_i v_i^T$ , which removes the already-found domin
50     current_matrix = matrix.copy().astype(float)
51     eigenvalues = []
52     eigenvectors = []
53
54     for _ in range(matrix.shape[0]):
55         eigenvalue, eigenvector, _ = power_method(current_matrix)
56         eigenvalues.append(eigenvalue)
57         eigenvectors.append(eigenvector)
58
59         v_column = eigenvector.reshape(matrix.shape[0], 1)
60         projector = matrix_multiply(v_column, transpose_matrix(v_column))
61         current_matrix = current_matrix - eigenvalue * projector
62
63     eigenvalues = np.array(eigenvalues)
64     eigenvectors = np.column_stack(eigenvectors)
65     order = np.argsort(eigenvalues)[::-1]
66     return eigenvalues[order], eigenvectors[:, order]
67
68
69 power_eigenvalues, power_eigenvectors = power_method_all(C)
70
71 print("All eigenvalues from the power method (largest to smallest):")
72 print(np.array2string(power_eigenvalues, precision=8, separator=", "))
73 print("\nAll eigenvectors from the power method (columns):")
74 print(np.array2string(power_eigenvectors, precision=8, separator=", "))
75
76 lambda_1 = power_eigenvalues[0]
77 v1 = power_eigenvectors[:, 0]
78 print("\nLargest eigenvalue:", f"{lambda_1:.8f}")
79 print("Corresponding eigenvector:")
80 print(np.array2string(v1, precision=8, separator=", "))
81

```

All eigenvalues from the power method (largest to smallest):
 [1.50422544e+01, 6.21210776e+00, 1.12396155e+00, 9.98621091e-01,
 -6.29432747e-24, -3.61254503e-08]

All eigenvectors from the power method (columns):
 [[0.13287204, 0.03618485, 0.49901362, -0.66816109, 0.49644621,
 -0.49644621],
 [0.23092481, 0.00815514, -0.32586398, 0.4445457 , -0.15892044,
 0.15892044],
 [-0.03490847, 0.21055279, 0.70893898, 0.53387526, 0.59427535,
 -0.59427535],
 [0.02798583, -0.34845616, 0.37442392, 0.26628587, 0.3250008 ,
 -0.3250008],
 [0.95851852, 0.09683514, 0.0204353 , -0.00268508, 0.51613112,
 -0.51613112],
 [-0.09088013, 0.90746511, -0.03990886, 0.00130353, -0.05572623,
 0.05572623]]

Largest eigenvalue: 15.04225443
 Corresponding eigenvector:
 [0.13287204, 0.23092481, -0.03490847, 0.02798583, 0.95851852,
 -0.09088013]

(c) Write a Python code to obtain the next largest eigenvalue λ_2 and its corresponding eigenvector v_2 by applying power method on $C - v_1 v_1^T C$. Having found out $v_1, v_2, \dots, v_{k-1}$, one can find λ_k and its corresponding eigenvector v_k by applying power method on $C - \sum_{j=1}^{k-1} v_j v_j^T C$. Give the code and also display the obtained eigenvalues and the corresponding eigenvectors. [1.5]

```

1 # Q2(3c): Obtain the next eigenpair by deflating the dominant direction and then applying the power method again.
2 # The matrix used for the second eigenpair is the deflated matrix  $C - \lambda_1 v_1 v_1^T$ , which avoids repeating the dominant
3 # This follows the general deflation rule: after finding eigenpairs  $(\lambda_1, v_1), \dots, (\lambda_k, v_k)$ , the next eigenpair is fo
4 # This is the numerical way of extracting the next largest eigen-direction while preserving orthogonality.
5

```

```

6 v1_column = v1.reshape(C.shape[0], 1)
7 projector_on_v1 = matrix_multiply(v1_column, transpose_matrix(v1_column))
8 deflated_from_v1 = C - lambda_1 * projector_on_v1
9
10 lambda_2, v2, iterations_2 = power_method(deflated_from_v1)
11
12 # print("Deflated matrix for the second eigenpair:")
13 # print(np.array2string(deflated_from_v1, precision=8, separator=", "))
14 print("\nSecond eigenvalue:", f"{lambda_2:.8f}")
15 print("Corresponding eigenvector:")
16 print(np.array2string(v2, precision=8, separator=", "))
17 print("Number of iterations for the second eigenpair:", iterations_2)
18
19 print("\nTop two eigenpairs from the power method:")
20 print("(λ1, v1) =", f"{lambda_1:.8f}", np.array2string(v1, precision=8, separator=", "))
21 print("(λ2, v2) =", f"{lambda_2:.8f}", np.array2string(v2, precision=8, separator=", "))

```

Second eigenvalue: 6.21210776
 Corresponding eigenvector:
 [0.03618485, 0.00815514, 0.21055279, -0.34845616, 0.09683514,
 0.90746511]
 Number of iterations for the second eigenpair: 7

Top two eigenpairs from the power method:
 (λ1, v1) = 15.04225443 [0.13287204, 0.23092481, -0.03490847, 0.02798583, 0.95851852,
 -0.09088013]
 (λ2, v2) = 6.21210776 [0.03618485, 0.00815514, 0.21055279, -0.34845616, 0.09683514,
 0.90746511]

(d) Find all the eigenvalues and eigenvectors using Python function and compare with the obtained result in (c). [0.5]

```

1 # Q2(3d): Use NumPy's symmetric eigensolver as a reference for the power-method eigenpairs.
2 # Only the eigenvalues and eigenvectors are shown; the difference values are not printed.
3 # This is a comparison step, not a new method. We want to verify whether the power method has correctly recovered the s
4 # NumPy returns eigenvalues in ascending order and stores the matching eigenvectors as columns.
5 eigenvalues_np, eigenvectors_np = np.linalg.eigh(C)
6
7 # Sort from largest to smallest to match the power-method presentation.
8 order = np.argsort(eigenvalues_np)[::-1]
9 eigenvalues_np = eigenvalues_np[order]
10 eigenvectors_np = eigenvectors_np[:, order]
11
12 # Align signs only so the eigenvectors can be compared visually.
13 # Since eigenvectors are determined up to sign, flipping the sign does not change the actual eigen-direction.
14 for i in range(C.shape[0]):
15     if np.dot(power_eigenvalues[:, i], eigenvectors_np[:, i]) < 0:
16         power_eigenvectors[:, i] = -power_eigenvectors[:, i]
17
18 print("Power-method eigenvalues (largest to smallest):")
19 print(np.array2string(power_eigenvalues, precision=8, separator=", "))
20 print("\nNumPy eigenvalues (largest to smallest):")
21 print(np.array2string(eigenvalues_np, precision=8, separator=", "))
22
23 print("\nPower-method eigenvectors (columns):")
24 print(np.array2string(power_eigenvectors, precision=8, separator=", "))
25 print("\nNumPy eigenvectors (columns):")
26 print(np.array2string(eigenvectors_np, precision=8, separator=", "))
27

```

Power-method eigenvalues (largest to smallest):
 [1.50422544e+01, 6.21210776e+00, 1.12396155e+00, 9.98621091e-01,
 -6.29432747e-24, -3.61254503e-08]

NumPy eigenvalues (largest to smallest):
 [1.50422544e+01, 6.21210772e+00, 1.12396187e+00, 9.98620729e-01,
 3.77522452e-15, 5.75461957e-16]

Power-method eigenvectors (columns):
 [[-0.13287204, 0.03618485, -0.49901362, 0.66816109, -0.49644621,
 -0.49644621],

```
[-0.23092481, 0.00815514, 0.32586398, -0.4445457, 0.15892044,
 0.15892044],
[ 0.03490847, 0.21055279, -0.70893898, -0.53387526, -0.59427535,
-0.59427535],
[-0.02798583, -0.34845616, -0.37442392, -0.26628587, -0.3250008,
-0.3250008 ],
[-0.95851852, 0.09683514, -0.0204353, 0.00268508, -0.51613112,
-0.51613112],
[ 0.09088013, 0.90746511, 0.03990886, -0.00130353, 0.05572623,
0.05572623]]
```

NumPy eigenvectors (columns):

```
[[-0.13287044, 0.03619598, -0.5000847, 0.6672603, 0.53448354,
 0.00645224],
[-0.23092445, 0.00818196, 0.32657507, -0.44395781, 0.80172531,
 0.00967836],
[ 0.03491781, 0.210542, -0.7080928, -0.53515488, -0.00492798,
 0.40821855],
[-0.02800128, -0.34845685, -0.37398023, -0.26695668, 0.00985596,
-0.81643709],
[-0.95851422, 0.09693785, -0.02044418, 0.00264718, -0.26724177,
-0.00322612],
[ 0.09092036, 0.9074557, 0.03986767, -0.00124151, 0.00492798,
-0.40821855]]
```

(e) Compare the number of iterations required by the power method to reach $10e-7$ accuracy.

```
1 # Q2(3e): Compare the number of iterations needed by the power method to reach
  1e-7 accuracy for every eigenvalue.
2 # The NumPy eigenvalues are already sorted from largest to smallest in the
  previous comparison step.
3 # The fix is to preserve that ordering and deflate only the already-processed
  larger eigen-directions.
4 # This is the iteration-count experiment: we test how many repetitions are
  needed for each eigen-direction to converge to the required tolerance.
5
6
7 def count_iterations_for_target(
8     matrix, actual_eigenvalue, tolerance=1e-7, max_iterations=10000
9 ):
10     # This helper computes how many power-method iterations are needed to
    approximate a target eigenvalue within a specified error.
11     # It starts from a unit vector, repeatedly applies the matrix, normalizes,
    and computes the Rayleigh quotient.
12     vector = np.ones((matrix.shape[0], 1))
13     vector = vector / vector_norm(vector)
14
15     for iteration in range(1, max_iterations + 1):
16         new_vector = matrix_multiply(matrix, vector)
17         new_vector_length = vector_norm(new_vector)
18         if new_vector_length == 0:
19             raise ValueError("The power method reached a zero vector.")
20         vector = new_vector / new_vector_length
21
22         transpose_vector = transpose_matrix(vector)
23         matrix_vector = matrix_multiply(matrix, vector)
24         rayleigh_quotient = matrix_multiply(transpose_vector, matrix_vector)
25         approximate_eigenvalue = rayleigh_quotient[0, 0]
26         error = abs(approximate_eigenvalue - actual_eigenvalue)
27
28         if error < tolerance:
29             return iteration
30
31     return max_iterations
32
33
34 # Keep the exact eigenvalues in the already-correct descending order from the
  NumPy comparison cell.
35 # Reversing them again causes the smaller eigenvalues to be checked first and
  makes the counts look wrong.
36 exact_eigenvalues_desc = eigenvalues_np.copy()
37 exact_eigenvectors_desc = eigenvectors_np.copy()
38
39 all_iterations = []
40 for mode_index in range(len(exact_eigenvalues_desc)):
41     target_eigenvalue = exact_eigenvalues_desc[mode_index]
42
43     # Deflate the already-processed larger eigen-directions only.
44     # This ensures the next iteration focuses on the current eigenvalue rather
    than re-approximating a previously found dominant direction.
45     deflated_matrix = C.copy().astype(float)
46     for previous_index in range(mode_index):
47         vector = exact_eigenvectors_desc[:, previous_index].reshape(C.shape[0],
```

```
47         1)
48     projector = matrix_multiply(vector, transpose_matrix(vector))
49     deflated_matrix = (
50         deflated_matrix - exact_eigenvalues_desc[previous_index] * projector
51     )
52
53     iterations_needed = count_iterations_for_target(deflated_matrix,
54                                                     target_eigenvalue)
55     all_iterations.append(iterations_needed)
56
57     print("Required accuracy:", 1e-7)
58     # print("Exact eigenvalues from NumPy (largest to smallest):")
59     # print(np.array2string(exact_eigenvalues_desc, precision=8, separator=", "))
60     print("\nIterations needed for each eigenvalue to reach 1e-7 accuracy:")
61     print(all_iterations)
```

Required accuracy: 1e-07

Iterations needed for each eigenvalue to reach 1e-7 accuracy:
[11, 6, 53, 1, 1, 1]

1 Start coding or [generate](#) with AI.